SEMANA 6 — EVIDENCIAS Y ADJUNTOS

ÍNDICE

Tarea 1 — Crear tabla ticket_attachments.....2
Tarea 2 — Preparar carpeta local de subida y rutas seguras.....3
Tarea 3 — Subida de adjuntos (solo Admin/Técnico)..... 6
Tarea 4 — Reglas de seguridad para adjuntos (mínimo).....7
Tarea 5 — Listado y descarga de adjuntos en el ticket.....8
Conclusión..... 10

Tarea 1 — Crear tabla ticket_attachments

Crear la tabla que almacenará los archivos adjuntos asociados a cada ticket, permitiendo que los tickets puedan contener evidencias como capturas, logs, PDFs, etc. Esta tabla es fundamental para la gestión de adjuntos en Secure Desk.
La tabla se ha denominado **attachments** y se ha definido con la siguiente estructura:

```php
// ==================================================
// TABLA ATTACHMENTS: archivos adjuntos a los tickets
// ==================================================
$db->exec("
   CREATE TABLE IF NOT EXISTS attachments (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        ticket_id INTEGER NOT NULL,
        filename TEXT NOT NULL,
        filepath TEXT NOT NULL,
        mime_type TEXT NOT NULL,
        filesize INTEGER,
        uploaded_by INTEGER,
        created_at TEXT NOT NULL DEFAULT (datetime('now')),
        FOREIGN KEY (ticket_id) REFERENCES tickets(id) ON DELETE CASCADE,
        FOREIGN KEY (uploaded_by) REFERENCES users(id) ON DELETE SET NULL
    );
");
```

attachments	
id	INTEGER
ticket_id	INTEGER
filename	TEXT
filepath	TEXT
mime_type	TEXT
filesize	INTEGER
uploaded_by	INTEGER
created_at	TEXT

Relaciones:

- **Con tickets**: **ticket_id** referencia a **tickets(id)** con ON DELETE CASCADE, lo que significa que si se elimina un ticket, todos sus adjuntos se eliminarán automáticamente.
- **Con usuarios**: **uploaded_by** referencia a **users(id)** con ON DELETE SET NULL, de modo que si un usuario es eliminado, el adjunto queda registrado pero sin asociación a un usuario (se muestra como "Usuario desconocido").

Tarea 2 — Preparar carpeta local de subida y rutas seguras

Se creó la carpeta **uploads/** en la raíz del proyecto. Para evitar mezclar archivos de diferentes tickets, se crea una subcarpeta por cada ticket. La estructura será **uploads/ticket_X/** donde X es el ID del ticket. Esto se hace dinámicamente en el script de subida.

Al subir un archivo (**public/uploads_attachment**) , el nombre original puede contener caracteres peligrosos (espacios, acentos, rutas). Por seguridad, se renombra internamente con un identificador único (hash aleatorio) manteniendo la extensión original. El nombre original se guarda en la base de datos para mostrarlo al usuario.

```php
// Generar nombre seguro (solo hash + extensión)
$extension = pathinfo($originalName, PATHINFO_EXTENSION);
$safeName = bin2hex(random_bytes(16)) . ($extension ? '.' . $extension : '');
```

Para la subida de archivos, como en todas las demás secciones, se verifica si está autentificado, si tienen rol admin para poder subir el archivo,...

Para proteger contra ataques de falsificación de petición, se incluye un token CSRF en el formulario y se verifica en el servidor. Se comprueba que el archivo se haya subido sin errores, que no exceda el tamaño y que su tipo MIME real esté en la lista blanca.

```php
if (!isset($_FILES['attachment']) || $_FILES['attachment']['error'] !== UPLOAD_ERR_OK) {
    die("Error al subir archivo.");
}

$file = $_FILES['attachment'];

// Validar tamaño
if ($file['size'] > MAX_FILE_SIZE) {
    die("El archivo excede el tamaño máximo de " . (MAX_FILE_SIZE / 1024 / 1024) . " MB.");
}

// Obtener tipo MIME real
$finfo = finfo_open(FILEINFO_MIME_TYPE);
$mime = finfo_file($finfo, $file['tmp_name']);
finfo_close($finfo);

if (!in_array($mime, ALLOWED_MIME_TYPES)) {
    die("Tipo de archivo no permitido.");
}
```

En caso de que no exista su carpeta, se crea la subcarpeta correspondiente al ticket.

```php
// Crear subcarpeta por ticket si no existe
$uploadDir = __DIR__ . '/../uploads/ticket_' . $ticket_id . '/';
if (!is_dir($uploadDir)) {
    mkdir($uploadDir, 0777, true); // permisos adecuados
}
```

Se guarda el nombre original, la ruta relativa, el tipo MIME, el tamaño y el usuario que subió el archivo y tras eso redirige a los detalles del tickets de nuevo.

```php
// Guardar en BD (incluyendo mime_type)
$stmt = $db->prepare("
    INSERT INTO attachments (ticket_id, filename, filepath, mime_type, filesize, uploaded_by)
    VALUES (:ticket_id, :filename, :filepath, :mime_type, :filesize, :uploaded_by)
");

$stmt->execute([
    ':ticket_id' => $ticket_id,
    ':filename' => $originalName,
    ':filepath' => 'ticket_' . $ticket_id . '/' . $safeName, // ruta relativa para guardar
    ':mime_type' => $mime,
    ':filesize' => $fileSize,
    ':uploaded_by' => $_SESSION['user_id']
]);
```

La vista ya fué preparada en la semana 3 (y la funcionalidad por mi parte, aunque en esta tarea la estoy mejorando y completando) como en todos los campos que se introduce texto o archivos, se hace la autenticación a través del ticket, y se muestra un listado con los tickets.

```php
<div class="attachments-list">
    <?php foreach ($attachments as $file): ?>
        <div class="attachment-card">
            <div class="file-info">
                <div class="file-name">📄 <?= htmlspecialchars($file['filename']) ?></div>
                <div class="file-meta">
                    <?= round($file['filesize'] / 1024, 2) ?> KB ·
                    <?= htmlspecialchars($file['created_at']) ?> ·
                    Subido por <?= htmlspecialchars($file['username'] ?? 'Desconocido') ?>
                </div>
            </div>
            <div class="file-actions">
                <a href="download_attachment.php?id=<?= $file['id'] ?>" class="btn-download">Descargar</a>
            </div>
        </div>
    <?php endforeach; ?>
```

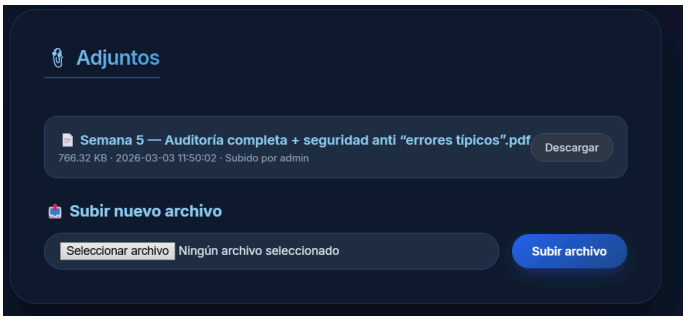
Para descargar el archivo (en **public/downloads_attachment**), se mirara que esté autenticado, aunque cualquier usuario podrá descargarlo, hasta los lectores. Tras eso, se obtiene el archivo desde la base de datos, a través de una consulta.

```php
$stmt = $db->prepare("SELECT * FROM attachments WHERE id = :id");
$stmt->execute([':id' => $id]);
$file = $stmt->fetch(PDO::FETCH_ASSOC);

if (!$file) {
    die("Archivo no encontrado.");
}
```

Tras eso, se envía a la vista el título, archivo, información... Para que se muestre se usa el tipo **MIME** almacenado para que el navegador lo maneje adecuadamente.

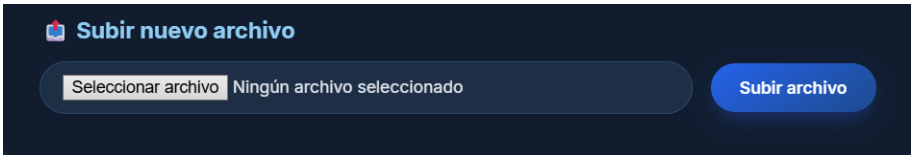
```php
// Enviar cabeceras
header('Content-Description: File Transfer');
header('Content-Type: ' . ($file['mime_type'] ?? 'application/octet-stream'));
header('Content-Disposition: attachment; filename="' . $file['filename'] . '"');
header('Content-Length: ' . filesize($path));
header('Cache-Control: private, max-age=0, must-revalidate');
header('Pragma: public');
```



Tarea 3 — Subida de adjuntos (solo Admin/Técnico)

Se añadió un formulario específico para la subida de archivos, ubicado en la sección de adjuntos. Este formulario solo se muestra si el usuario autenticado no es lector.

```php
<!-- Formulario de subida (solo admin/técnico) -->
<?php if ($_SESSION['role'] !== 'lector'): ?>
    <h3 style="margin: 25px 0 15px; color: #90cdf4;">📤 Subir nuevo archivo</h3>
    <form action="upload_attachment.php" method="POST" enctype="multipart/form-data" class="upload-form">
        <!-- Campo CSRF añadido -->
        <input type="hidden" name="csrf_token" value="<?= htmlspecialchars($csrf_token) ?>">
        <input type="hidden" name="ticket_id" value="<?= $ticket['id'] ?>">
        <input type="file" name="attachment" required class="file-input">
        <button type="submit" class="btn-primary">Subir archivo</button>
    </form>
<?php endif; ?>
```

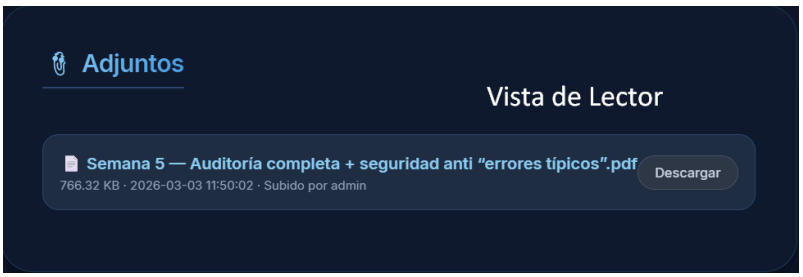


Como ya antes se mostró la parte lógica también está hecha, que sería **public/uploads_attachment** en este script, se ejecutan las siguientes tareas:

- Verifica que el usuario esté autenticado y no sea lector.
- Comprueba el token CSRF.
- Valida que el archivo se haya subido correctamente.
- Aplica restricciones de seguridad (tamaño máximo y tipos MIME permitidos).
- Genera un nombre seguro y mueve el archivo a la carpeta uploads/ticket_X/.
- Inserta un registro en la tabla attachments con toda la información.
- Redirige de vuelta al detalle del ticket.

Todas estas acciones se han mostrado anteriormente en la tarea 2. Por lo tanto, como se ha explicado el bloqueo se ha ejecutado tanto en la vista como en la lógica para los lectores:

- **En la vista**: el formulario solo se renderiza si **$_SESSION['role'] !== 'lector'**. Así, los lectores no ven la opción de subir archivos.
- **En el controlador**: al inicio de **upload_attachment.php** se verifica que el usuario no sea lector. Si un lector intentara acceder directamente a este script (por ejemplo, escribiendo la URL), recibiría el mensaje "No autorizado".



Tarea 4 — Reglas de seguridad para adjuntos (mínimo)

En esta tarea pasamos a configurar y filtrar los tipos de archivos que se pueden enviar y cuáles no, por lo tanto en **public/uploads_attachment** se definen constantes para el tamaño máximo que en nuestro caso son 5MB y los tipos MIME permitidos (pueden estar en un archivo de configuración global o al inicio del script, en nuestro caso al inicio del script).

```php
// Configuración de subida
define('MAX_FILE_SIZE', 5 * 1024 * 1024); // 5 MB
define('ALLOWED_MIME_TYPES', [
    'image/jpeg', 'image/png', 'image/gif',
    'application/pdf', 'text/plain',
    'application/msword', 'application/vnd.openxmlformats-officedocument.wordprocessingml.document'
]);
```

Tras esto, se valida que efectivamente ocupa el espacio y tiene MIME necesario para poder subirse.

```php
// Validar tamaño
if ($file['size'] > MAX_FILE_SIZE) {
    die("El archivo excede el tamaño máximo de " . (MAX_FILE_SIZE / 1024 / 1024) . " MB.");
}

// Obtener tipo MIME real del archivo
$finfo = finfo_open(FILEINFO_MIME_TYPE);
$mime = finfo_file($finfo, $file['tmp_name']);
finfo_close($finfo);

// Validar tipo MIME
if (!in_array($mime, ALLOWED_MIME_TYPES)) {
    die("Tipo de archivo no permitido.");
}
```

Y en caso de que no se pueda, como se muestra en el código, enseñará un error ya sea por el tamaño o por el MIME.

Tarea 5 — Listado y descarga de adjuntos en el ticket

En el controlador **public/ticket_detail.php**, después de cargar los datos del ticket, se realiza una consulta a la base de datos para recuperar todos los archivos adjuntos de ese ticket, incluyendo el nombre del usuario que los subió (para mostrarlo en la vista).

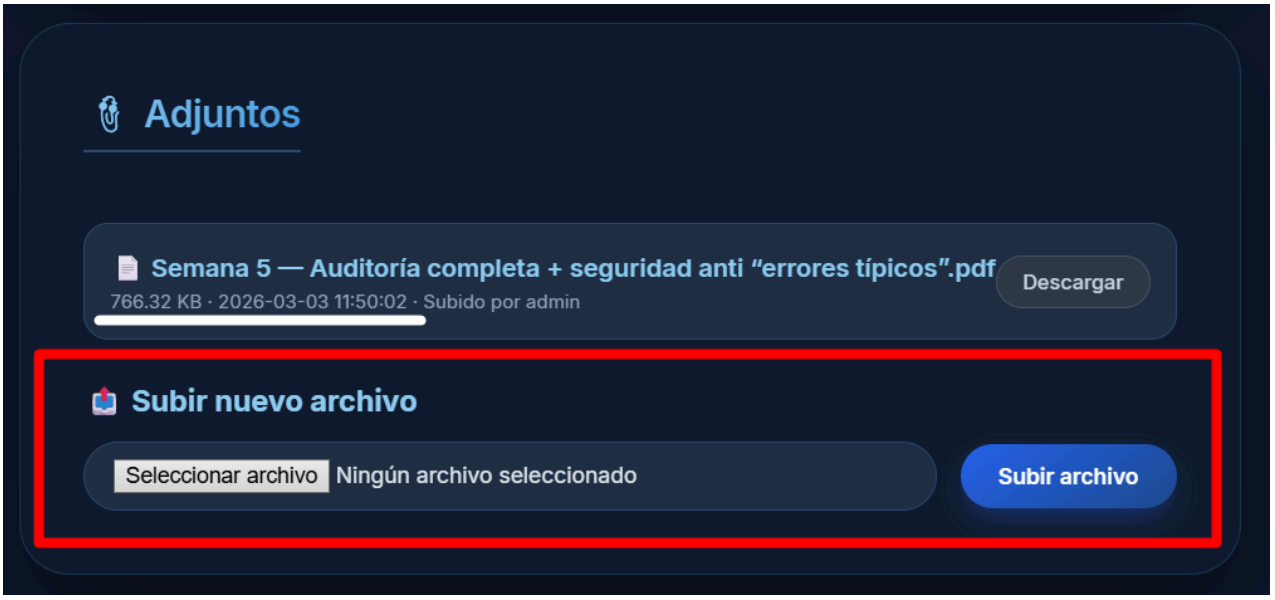
```php
// Obtener adjuntos
$stmt = $db->prepare("
    SELECT a.*, u.username
    FROM attachments a
    LEFT JOIN users u ON a.uploaded_by = u.id
    WHERE a.ticket_id = :ticket_id
    ORDER BY a.created_at DESC
");
$stmt->execute([':ticket_id' => $id]);
$attachments = $stmt->fetchAll(PDO::FETCH_ASSOC);
```

En **views/tickets/detail.php** se añade un bloque que recorre el array **$attachments** y muestra cada archivo en una tarjeta con su información y un enlace de descarga.

```php
<div class="detail-card">
    <h2 class="section-title">📎 Adjuntos</h2>

    <?php if (empty($attachments)): ?>
        <p style="color: #94a3b8;">No hay archivos adjuntos.</p>
    <?php else: ?>
        <div class="attachments-list">
            <?php foreach ($attachments as $file): ?>
                <div class="attachment-card">
                    <div class="file-info">
                        <div class="file-name">📄 <?= htmlspecialchars($file['filename']) ?></div>
                        <div class="file-meta">
                            <?= round($file['filesize'] / 1024, 2) ?> KB ·
                            <?= htmlspecialchars($file['created_at']) ?> ·
                            Subido por <?= htmlspecialchars($file['username'] ?? 'Desconocido') ?>
                        </div>
                    </div>
                    <div class="file-actions">
                        <a href="download_attachment.php?id=<?= $file['id'] ?>" class="btn-download">Descargar</a>
                    </div>
                </div>
            <?php endforeach; ?>
        </div>
    <?php endif; ?>
```

Como se puede ver en la siguiente imagen se mostrará una carta donde primero se muestra el título del archivo, abajo en pequeño salen todos los datos como el tamaño, fecha de subida y hora, incluso quien lo ha subido, lo rodeado por un cuadrado rojo es la subida de archivos, lo cual solo es visible en caso de ser un usuario administrador.



Conclusión

Ahora los tickets pueden contener evidencias: capturas de pantalla, archivos de log, PDFs, documentos, etc. Hemos implementado un sistema completo de adjuntos que incluye desde la organización física en carpetas separadas por ticket (**uploads/ticket_X/**) hasta la validación estricta de tipos de archivo (solo imágenes, PDFs y textos) y límite de tamaño (5 MB), garantizando así la seguridad y evitando sorpresas desagradables.

Además de que se cuida aun la seguridad, se muestran todos los datos de los adjuntos, esta semana ha sido fácil de hacer ya que gran parte de la estructura ya la tenía de antes de la semana 3, aunque se me ha hecho difícil de documentar a la vez por eso mismo y he tenido que repetir conceptos....

Código completo en el GitHub:
https://github.com/elgancho06/Practicas-DAM